

Rapport TER

Approche moderne du rendu différentiel

2026

Corentin VAILLANT, Julien LEFEBVRE

Université de Toulouse

2026

L3 MIDL

Encadrant : Mathias PAULIN

Table des matières

1	Introduction	3
2	Le Rendu	4
2.1	L'équation du rendu	4
2.2	Le Ray Tracing	4
2.3	Le Path Tracing	5
2.4	Autres Méthodes	5
3	La Différentiation	6
3.1	La Méthode Naïve	6
3.2	La Méthode Etudiée (Méthode de l'UC)	6
3.3	Autres Méthodes (Methode de l'EPFL)	6
4	Implementation du Path Tracer	7
5	Difficultés Rencontrées	8
6	Conclusion	9
7	Bibliographie	10

1 | Introduction

Le rendu différentiel est une branche de l'informatique graphique où l'on cherche à trouver la différentielle de l'image d'un rendu d'une scène, ce qui est très utile dans de nombreux domaines. Il possède de nombreuses applications, notamment dans le rendu plus classique, car il permet de faire varier des paramètres de la scène sans obligation de rendre la scène une nouvelle fois de zéro, ce qui peut être très coûteux. Il est aussi utile dans le rendu physiquement réaliste parce qu'il a permis de résoudre des problèmes d'analyse-par-synthèse dans des domaines comme le rendu de vêtements ou encore la création de matériaux translucides. Son utilité ne se limite pas qu'au rendu, il possède aussi des applications dans le domaine du machine learning car il permet d'entraîner des réseaux de neurones de manière plus efficace. Le rendu différentiel a une communauté de recherche très active. En effet ce domaine est très challengeant parce qu'à ce jour aucun algorithme efficace n'introduisant pas de biais n'a été découvert. Cela est notamment dû au fait de l'absence d'estimateurs de Monte Carlo efficaces.

Dans ce TER, nous nous sommes intéressés aux travaux de Cheng Zhang, Bailey Miller, Kai Yan, Ioannis Gkioulekas et Shuang Zhao (**Path-Space Differentiable Rendering**) qui proposent une solution sans biais à ce problème basée sur la séparation en deux sous-problèmes.

Pour effectuer ce TER, nous nous sommes aussi intéressés au ray tracing ainsi qu'au path tracing, notamment grâce à la série de livres **Raytracing in one weekend**, dans le but d'acquérir les bases nécessaires à la compréhension du papier.

Dans le cadre de ce travail, nous avons aussi implémenté un Path Tracer sur GPU avec Vulkan (**GitHub du projet**).

Nous allons introduire dans un premier temps le ray tracing ainsi que ses limites. Puis nous regarderons ce qu'est le Path Tracing et comment il corrige les limites du Ray Tracing. Dans un troisième temps, nous parlerons du rendu différentiel et de la méthode proposée par Shuang Zhao et son équipe. Enfin nous finirons par regarder une autre approche à ce problème avec les travaux de Tizian Zeltner, Sébastien Speierer, Iliyan Georgiev et Wenzel Jakob (**Monte Carlo Estimators for Differential Light Transport**).

2 | Le Rendu

2.1 | L'équation du rendu

Pour effectuer un rendu 3D, nous allons essayer de calculer la luminance des différentes surfaces de notre scène. La luminance représente la luminosité que l'œil humain perçoit, provenant d'une surface qui émet de la lumière soit en tant que source de lumière, soit par réflexion ou transmission. L'équation du rendu (**Page Wikipedia Equation du Rendu**) nous permet de calculer la luminance sur les différentes surfaces de notre scène 3D. On peut la définir comme ci-dessous :

$$L_o(x, \omega_o) = L_e(x, \omega_o) + \int_{\Omega} L_i(x, \omega_i) f_i(x, \omega_0, \omega_i) \overline{\cos}(\theta_i) d\omega_i \quad (1)$$

- $L_o(x, \omega_o)$ représente la luminance sortante au point x dans la direction ω_o .
- $L_e(x, \omega_o)$ représente la luminance émise par le point x dans la direction ω_o .
- Ω représente la sphère de rayon 1 autour du point x .
- $L_i(x, \omega_i)$ représente la luminance arrivant en x depuis la direction ω_i et elle est définie de la manière suivante : $L_i(x, \omega_i) = L_o(vis(x, \omega_i), -\omega_i)$ où $vis(x, \omega_i)$ donne le premier point intersecté en partant dans la direction ω_i à partir du point x .
- $f_i(x, \omega_0, \omega_i)$ représente la BRDF qui indique comment la luminance arrivant en x depuis la direction ω_i et réfléchi dans la direction ω_o .
- θ représente l'angle formé ω_i et la $\vec{n}$ normale de la surface.
- $\overline{\cos}(\theta) = \cos(\theta)$ si $\cos(\theta) > 0$ sinon 0.

L'un des problèmes de cette équation est que l'on ne peut pas résoudre l'intégrale de façon analytique, notamment en raison de sa nature récursive. C'est pour cela que des méthodes pour estimer cette intégrale ont été mises en place.

2.2 | Le Ray Tracing

Le ray tracing (lancée de rayon dans la langue de Molière) est un algorithme permettant d'estimer l'équation du rendu (eq. 1). Il se base sur une technique de lancer de rayon à partir de la caméra vers la scène, c'est-à-dire dans le sens inverse de la lumière. Cela donne le même résultat que dans le sens de la lumière d'après le principe du retour inverse de la lumière de Fermat. Il se base aussi sur les lois de Snell-Decartes de réflexion et réfraction de la lumière. Une méthode de ray tracing naïve de la marche aléatoire :

Algorithm 1 . – Marche Aléatoire

```

1: procedure Li(Rayon r, depth  $\in \mathbb{N}$ )
2:   intersection  $\leftarrow$  Intersection(r)
3:    $\triangleright$  Cas où le rayon n'intersecte aucune surface
4:   if Pas d'intersection then
5:      $L_e \leftarrow 0$ 
6:     for lumière  $\in$  Lumières Directionelles do
7:        $L_e \leftarrow L_e + \text{lumière}.L_e$ 
8:     end
9:     return  $L_e$ 
10:  end
11:   $\triangleright$  Recupération de la surface intersectée et de son émission
12:  surface  $\leftarrow$  intersection.surface
13:   $\omega_o \leftarrow -r.\text{direction}$ 
14:   $L_e \leftarrow \text{surface}.Le(\omega_o)$ 
15:   $\triangleright$  Cas où on a atteint le nombre maximum d'itération
16:  if depth = maxDepth then
17:    return  $L_e$ 
18:  end
19:   $\triangleright$  Calcul de la partie non récursive de l'intégrale de l'équation du rendu
20:  bsdf  $\leftarrow$  surface.BSDF
21:   $\omega_i \leftarrow \mathcal{U}(\Omega)$ 
22:   $f_{\cos} \leftarrow \text{bsdf}(\omega_o, \omega_i) \times |\omega_i \cdot \vec{n}|$ 
23:   $\triangleright$  Si la partie non récursive est nulle il est inutile de continuer d'itérer
24:  if  $f_{\cos} = 0$  then
25:    return  $L_e$ 
26:  end
27:   $\triangleright$  Appel récursif
28:  r  $\leftarrow$  CreerRayon( $\omega_i$ )
29:  return  $L_e + f_{\cos} \times \text{Li}(r, \text{depth}+1) / (1 / (4 \times \pi))$ 
30: end

```

Il est possible d'améliorer cette algorithme en introduisant

2.3 | Le Path Tracing

2.4 | Autres Méthodes

3 | La Différentiation

3.1 | La Méthode Naïve

3.2 | La Méthode Etudiée (Méthode de l'UC)

3.3 | Autres Méthodes (Méthode de l'EPFL)

4 | Implementation du Path Tracer

5 | Difficultés Rencontrées

6 | Conclusion

7 | Bibliographie